Tradays
Экономический календарь
Добавлены новости Италии
>>

Логин mql5.com для metatrader.com
Узна
боль

Алгоритм опыления цветов — Flower Pollination Algorithm (FPA)

0 Andrey Dik 89

Содержание

1. Введение
2. Реализация алгоритма
3. Результаты тестов
4. Выводы

Введение

FPA (Flower Pollination Algorithm) — один из самых известных метаэвристических алгоритмов последнего десятилетия. Алгоритм был предложен Xin-She Yang и опубликован в 2012 г. У него много модификаций, а описание укладывается в две формулы и одну монетку. Именно поэтому он интересен трейдеру: это тот алгоритм, который скорее всего попадётся под руку, и его скорее всего запустят с параметрами по умолчанию.

Вот здесь и начинается проблема. С авторскими параметрами FPA на нашем стенде показывает 26% — уровень, близкий к случайному поиску. Трейдер, который подключил такой оптимизатор к своему советнику, получит набор параметров, найденный почти наугад, и будет считать его результатом оптимизации.

В этой статье мы перенесём FPA в MQL5 и разберёмся, какая из двух его операций действительно работает, а какая только расходует бюджет. Посмотрим, какими параметрами алгоритм поднимается с 26% до среднего уровня большинства алгоритмов. Отдельно разберём, почему в больших размерностях не помогает уже никакая настройка. Отдельно проверим, что улучшение честное: что оно не держится на особенностях тестового стенда.

Реализация алгоритма

Автор строит алгоритм на двух способах опыления. При перекрёстном опылении пыльцу разносят насекомые и птицы, и она может улететь далеко. При самоопылении пыльца переносится ветром или внутри одного растения и остаётся рядом. Каждый цветок считается одним решением задачи, а популяция — набором таких решений.

Из этой метафоры автор выводит четыре правила:

1. Перекрёстное опыление — это глобальный поиск, а перелёты опылителей подчиняются распределению Леви.
2. Самоопыление — это локальный поиск.
3. Вероятность опыления между двумя цветками тем выше, чем они более похожи (постоянство цветков).

4. Выбор между глобальным и локальным опылением задаёт вероятность переключения p с небольшим перевесом в сторону локального.

Глобальное опыление в статье и в коде автора записано по-разному, статья:

```
x_i(t+1) = x_i(t) + L · (g - x_i(t)),   L > 0 – длина шага из распределения Леви
```

код:

```
x_i(t+1) = x_i(t) + L · (x_i(t) - g),   L – симметричное число по алгоритму Мантеньи
```

Здесь g — лучшее найденное решение, а L — случайный множитель из распределения Леви, свой для каждой координаты. Распределение Леви почти всегда даёт маленькие значения, но изредка очень большие. Поэтому цветок чаще всего смещается немного, а иногда улетает далеко. В коде автора L строится алгоритмом Мантеньи с индексом β = 1.5 и умножается на 0.01:

```
σ = ( Γ(1+β)·sin(πβ/2) / ( Γ((1+β)/2)·β·2^((β-1)/2) ) )^(1/β) ≈ 0.6966 при β = 1.5 ι
```

В статье L описан как положительная длина шага, и тогда формула действительно ведёт цветок к лучшему решению. В коде, который мы переносим, алгоритм Мантеньи даёт симметричное L: положительные и отрицательные значения выпадают одинаково часто. Поэтому знак разности ничего не меняет. Значит, по каждой координате цветок с равной вероятностью приближается к лучшему или удаляется от него. В такой реализации лучшее решение задаёт не направление шага, а его размах. Чем дальше цветок от лидера по какой-то координате, тем сильнее его встряхивает по этой координате. Сам лидер глобальным шагом не двигается вовсе: для него x_i − g = 0.

Локальное опыление использует два случайных цветка j и k из популяции:

```
x_i(t+1) = x_i(t) + ε · (x_j(t) - x_k(t)), ε ~ U(0, 1)
```

Случайное число ε одно на весь вектор, поэтому цветок сдвигается строго вдоль направления от k к j, на долю этого расстояния. Это та же операция, что и в дифференциальной эволюции, только без кроссовера: меняются сразу все координаты.

Дальше всё просто. На каждой итерации каждый цветок бросает монетку и делает один из двух шагов. Новое положение принимается, только если оно не хуже старого, иначе цветок остаётся на месте. Лучшее решение обновляется по ходу дела. Вот и весь алгоритм: два оператора, один переключатель и жадный отбор.

На рисунке 1 оба шага показаны на плоскости. Слева — облако пробных точек глобального шага: оно симметрично вокруг цветка, а не вытянуто в сторону лидера, и дальние выбросы идут вдоль одной оси. Справа — локальный шаг: все пробы лежат на одном отрезке, параллельном разности двух случайных цветков.

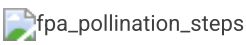


Рисунок 1. Два способа получить пробную точку в FPA: глобальное опыление (слева) — встряска вокруг цветка, локальное (справа) — сдвиг вдоль разности двух соседей

Слева на рисунке: глобальное опыление (выполняется, когда rand > pSwitch): s_i = x_i + levyScale · L ⊙ (x_i − g), где для каждой координаты берётся своё число Леви L. Пунктирный прямоугольник показывает масштаб шага: ± levyScale · |x_i − g| по каждой координате. Распределение L симметрично, поэтому это встряска вокруг x_i, а не движение к лучшему решению g. Редкие дальние прыжки идут вдоль отдельных осей, а лучший цветок (x_i = g) этим шагом не сдвигается.

Справа на рисунке: локальное опыление (выполняется, когда rand ≤ pSwitch): s_i = x_i + ε · (x_j − x_k), где одно ε ~ U(0, 1) действует на все координаты. Все пробные точки лежат на одном отрезке, параллельном

x_j − x_k. Направление и размер шага задают два случайных цветка; в начале поиска шаг охватывает большую часть области сразу по всем координатам.

Наша реализация следует коду автора и выглядит следующим образом:

```
Вход: n (размер популяции), pSwitch (вероятность локального шага),
      levyScale (масштаб шага Леви), β = 1.5

1. Инициализация
   для каждого цветка i:
      x_i = случайная точка в области поиска
      f_i = F(x_i)
   g = лучший из x_i

2. Главный цикл, пока не исчерпан бюджет
   для каждого цветка i:
      если rand > pSwitch:                          // глобальное опыление
         для каждой координаты c:
            L = levyScale · Levy(β)                  // свой L на каждую координату
            s_i[c] = x_i[c] + L · (x_i[c] - g[c])
      иначе:                                         // локальное опыление
         ε = rand(0, 1)                                // один на весь вектор
         j, k = два разных случайных цветка
         s_i = x_i + ε · (x_j - x_k)
      прижать s_i к границам области поиска
      f_new = F(s_i)
      если f_new не хуже f_i:   x_i = s_i,  f_i = f_new
      если f_new не хуже F(g): g = s_i

3. Результат: g

Levy(β), алгоритм Мантеньи:
   u = N(0,1) · σ,   v = N(0,1)
   вернуть u / |v|^(1/β)
```

Код и статья расходятся в четырёх местах. Самое важное — условие переключения, от него зависит соотношение двух шагов.

Первое — условие переключения. В статье глобальный шаг выполняется при rand < p, а в коде — при rand > p. В итоге при p = 0.8 статья делает 80% глобальных шагов, а код — 20%. Код при этом согласуется с правилом 4 (перевес в сторону локального опыления), а псевдокод статьи — не согласуется.

Второе — база локального шага. В статье шаг строится от принятого решения цветка x_i, а в коде — от последнего пробного вектора S(i). После отклонённого шага код стартует от плохой точки, и локальные шаги превращаются в блуждание, оторванное от отбора.

Третье — масштаб шага Леви. В статье множитель γ не задан, в коде он равен 0.01. В нашей реализации это параметр levyScale.

Четвёртое — направление глобального шага. В статье длина шага L положительна и умножается на (g − x_i), так что цветок движется к лучшему решению. В коде L симметрично (алгоритм Мантеньи) и умножается на (x_i − g), поэтому знак шага случаен и глобальный шаг оказывается встряской вокруг цветка, а не движением к лидеру. Наша реализация следует коду.

Есть и *пятое* отличие, уже между кодом автора и нашим стендом. Автор обновляет лучшее решение сразу после каждого цветка, и следующий цветок в той же итерации уже видит нового лидера. На стенде все цветки одной эпохи оцениваются вместе, и лидер обновляется один раз в конце эпохи. На результат это заметно не влияет, но для точности об этом стоит знать.

В нашей реализации база локального шага вынесена в переключатель fromCode, а условие оставлено как в коде. Вариант статьи при этом воспроизводится без лишнего флага: (rand < 0.8 → глобальный) — то же самое, что (rand > 0.2 → глобальный), то есть pSwitch = 0.2 и fromCode = 0.

Наша реализация в коде. Алгоритм реализован классом C_AO_FPA, наследником базового C_AO. Одна итерация FPA равна одной эпохе стенда: каждый цветок делает один шаг и получает одну оценку. Отдельный массив структур не понадобился: всё состояние цветка укладывается в поля агента, которые уже есть в базовом классе.

SetParams переносит параметры из массива params в поля класса и ограничивает их допустимыми значениями. Популяция не может быть меньше двух цветков, иначе локальному шагу не из чего взять два разных партнёра. Исправленные значения записываются обратно в params, чтобы строка с параметрами в журнале совпадала с тем, что реально запущено. У алгоритма четыре внешних параметра. Значения по умолчанию — лучшая конфигурация, найденная в тестах.

```
//+------------------------------------------------------------------+
//|                                                                  |
//+------------------------------------------------------------------+
class C_AO_FPA : public C_AO
  {
public:
                     ~C_AO_FPA() {}
                      C_AO_FPA()
     {
      ao_name = "FPA";
      ao_desc = "Flower Pollination Algorithm";
      ao_link = "https://www.mql5.com/ru/articles/25138";

      popSize   = 10;     // размер популяции
      pSwitch   = 0.2;    // вероятность локального шага
      levyScale = 0.5;    // масштаб шага Леви
      fromCode  = false;  // база локального шага

      ArrayResize(params, 4);
      params [0].name = "popSize";
      params [0].val = popSize;
      params [1].name = "pSwitch";
      params [1].val = pSwitch;
      params [2].name = "levyScale";
      params [2].val = levyScale;
      params [3].name = "fromCode";
      params [3].val = fromCode ? 1.0 : 0.0;
     }

   void               SetParams()
     {
      popSize   = (int)params [0].val;
      pSwitch   = params [1].val;
      levyScale = params [2].val;
      fromCode  = params [3].val != 0.0;

      if(popSize < 2)
         popSize = 2;   // локальному шагу нужны два разных агента
      if(pSwitch < 0.0)
         pSwitch = 0.0;
      if(pSwitch > 1.0)
         pSwitch = 1.0;
```

```
      if(levyScale < 0.0)
         levyScale = 0.0;

      params [0].val = popSize;
      params [1].val = pSwitch;
      params [2].val = levyScale;
      params [3].val = fromCode ? 1.0 : 0.0;
     }

  bool               Init(const double &rangeMinP  [],
                          const double &rangeMaxP  [],
                          const double &rangeStepP [],
                          const int     epochsP = 0);

  void               Moving();
  void               Revision();

  //------------------------------------------------------------------
  double             pSwitch;     // вероятность локального опыления
  double             levyScale;   // масштаб шага Леви
  bool               fromCode;    // база локального шага: true – a[i].c, false – a[i]

private:
  void               GlobalPollination(int i);
  void               LocalPollination(int i);

  double             Levy();
  double             Gauss();
  double             RndOpen();
 };
```

Init сводится к стандартной инициализации базового класса. Своих массивов и расписаний у алгоритма нет, а σ для β = 1.5 посчитано заранее и зашито константой.

```
//+------------------------------------------------------------------+
//|                                Init                              |
//+------------------------------------------------------------------+
bool C_AO_FPA::Init(const double &rangeMinP  [],
                    const double &rangeMaxP  [],
                    const double &rangeStepP [],
                    const int     epochsP = 0)
 {
  if(!StandardInit(rangeMinP, rangeMaxP, rangeStepP))
     return false;

  return true;
 }
```

Генератор шагов Леви. Сам шаг Леви строится из двух независимых нормальных чисел, которые даёт преобразование Бокса — Мюллера. Здесь есть две тонкости, связанные с тем, что стандартный генератор MQL5 выдаёт целые числа от 0 до 32767. Во-первых, логарифм нуля не определён, поэтому RndOpen перевыбирает нулевое значение. Во-вторых, RNDprobab иногда возвращает ровно 1.0, и тогда нормальное число равно нулю. Для числителя это безвредно, а для знаменателя означало бы деление на ноль, поэтому v перевыбирается.

```
//+------------------------------------------------------------------+
//| равномерное на (0; 1]                                            |
//+------------------------------------------------------------------+
double C_AO_FPA::RndOpen()
  {
   double r;
   do
      r = u.RNDprobab();
   while(r <= 0.0);
   return r;
  }
```

Преобразование Бокса — Мюллера.

```
//+------------------------------------------------------------------+
//| N(0,1), Box-Muller                                               |
//+------------------------------------------------------------------+
double C_AO_FPA::Gauss()
  {
   return sqrt(-2.0 * log(RndOpen())) * cos(2.0 * M_PI * u.RNDprobab());
  }
```

Алгоритм Мантеньи. При 15-битном генераторе наименьшее ненулевое |v| около $4 \cdot 10^{-7}$, поэтому множитель |v|^(−2/3) не превышает примерно $2 \cdot 10^{4}$. Тяжёлый хвост распределения сохраняется, а бесконечностей не возникает.

```
//+------------------------------------------------------------------+
//| шаг Леви, алгоритм Мантенья: u / |v|^(1/beta)                   |
//| v = 0 возможно (RndOpen = 1 даёт нулевой радиус) – перевыборка   |
//+------------------------------------------------------------------+
double C_AO_FPA::Levy()
  {
   double uu = Gauss() * 0.6965745025576968;
   //double uu = Gauss() * FPA_SIGMA;
   double v;
   do
      v = Gauss();
   while(v == 0.0);

   return uu / pow(fabs(v), 1.0 / 1.5);
   //return uu / pow(fabs(v), 1.0 / FPA_BETA);
  }
```

Глобальное опыление. Метод строит пробную точку от принятого решения цветка. Для каждой координаты берётся своё число Леви, и размах сдвига по этой координате пропорционален расстоянию до лучшего решения. Вышедшая за границу координата прижимается к ней, как в функции simplebounds автора.

```
//+------------------------------------------------------------------+
//| глобальное опыление: s_i = x_i + levyScale * L (.) (x_i - g)     |
//+------------------------------------------------------------------+
void C_AO_FPA::GlobalPollination(int i)
  {
   double x;

   for(int c = 0; c < coords; c++)
```

```
      {
       x = a [i].cB [c] + levyScale * Levy() * (a [i].cB [c] - cB [c]);
       a [i].c [c] = u.SeInDiSp(x, rangeMin [c], rangeMax [c], rangeStep [c]);
      }
  }
```

Локальное опыление. Метод выбирает два разных случайных цветка и сдвигает базу на долю их разности. Как и в коде автора, сам цветок i не исключается из выбора: в оригинале партнёры берутся из randperm(n), и i там может оказаться. База шага задаётся переключателем fromCode.

```
//+------------------------------------------------------------------+
//| локальное опыление: s_i = base_i + eps * (x_j - x_k)             |
//+------------------------------------------------------------------+
void C_AO_FPA::LocalPollination(int i)
  {
   double eps = u.RNDprobab();

   int j = u.RNDminusOne(popSize);
   int k;
   do
      k = u.RNDminusOne(popSize);
   while(k == j);

   double base, x;

   for(int c = 0; c < coords; c++)
     {
      base = fromCode ? a [i].c [c] : a [i].cB [c];
      x    = base + eps * (a [j].cB [c] - a [k].cB [c]);
      a [i].c [c] = u.SeInDiSp(x, rangeMin [c], rangeMax [c], rangeStep [c]);
     }
  }
```

Moving. На первой эпохе цветки разбрасываются равномерно по всей области поиска. На каждой следующей каждый цветок бросает монетку и делает один из двух шагов. Условие записано как в коде автора: глобальный шаг выполняется, когда случайное число больше pSwitch.

```
//+------------------------------------------------------------------+
//|                              Moving                              |
//+------------------------------------------------------------------+
void C_AO_FPA::Moving()
  {
//--- первый прогон: стартовая популяция
   if(!revision)
     {
      for(int i = 0; i < popSize; i++)
         for(int c = 0; c < coords; c++)
            a [i].c [c] = u.SeInDiSp(u.RNDfromCI(rangeMin [c], rangeMax [c]), rangeMin
      return;
     }

//--- одна итерация FPA = одна эпоха стенда
   for(int i = 0; i < popSize; i++)
     {
      if(u.RNDprobab() > pSwitch)
         GlobalPollination(i);
```

```
      else
         LocalPollination(i);
      }
   }
```

Revision. После оценки каждый цветок сравнивает пробную точку со своим принятым решением и переходит туда, если она не хуже. Равные значения принимаются, как и в оригинале (Fnew <= Fitness), и это позволяет цветку дрейфовать по плоским участкам, что полезно на ступенчатой Megacity. Стенд максимизирует, поэтому знак сравнения развёрнут. На первом проходе все a[i].fB равны −DBL_MAX, так что стартовая популяция принимается целиком без отдельной ветки. Обновление принятых решений в Revision — самая важная строка всей реализации. Без неё поля a[i].cB остаются нулевыми, и оба оператора начинают строить пробы от начала координат. Такой алгоритм всё ещё что-то находит, но уже за счёт положения начала координат относительно области поиска, а не за счёт популяции.

```
//+------------------------------------------------------------------+
//|                            Revision                              |
//+------------------------------------------------------------------+
void C_AO_FPA::Revision()
  {
//--- сохранить лучшее решение
   for(int i = 0; i < popSize; i++)
     {
      if(a [i].f >= a [i].fB)
        {
         a [i].fB = a [i].f;
         ArrayCopy(a [i].cB, a [i].c, 0, 0, coords);
        }
      if(a [i].f >= fB)
        {
         fB = a [i].f;
         ArrayCopy(cB, a [i].c, 0, 0, coords);
        }
     }

//--- первый проход: a[i].fB = -DBL_MAX, стартовая популяция принята целиком
   if(!revision)
      revision = true;
  }
```

Результаты тестов

Все прогоны выполнены на стандартном стенде серии: функции Hilly, Forest и Megacity в трёх размерностях (5, 25 и 500 функций, то есть 10, 50 и 1000 координат), 10 000 вычислений функции и 10 повторов.

Отдельной оценки разброса для FPA мы не делали. По повторным прогонам других алгоритмов серии на этом же стенде разброс итоговой оценки составляет порядка 1–2 процентных пунктов, поэтому меньшие различия мы не считаем значимыми.

Параметры подбирались на том же стенде, на котором измеряется итог. Композитный тест и стенд со сдвигом служат проверкой на других данных. В таблицах результаты трёх функций усреднены по каждой размерности: так лучше видно, где именно рождается разница.

От авторских параметров к рабочим. Начали мы с двух канонических вариантов: точной копии кода автора и варианта псевдокода статьи. Оба оказались слабыми. Причина — множитель 0.01 в шаге Леви:

с ним глобальный шаг сдвигает цветок в среднем на сотые доли расстояния до лидера и практически неподвижен. Увеличение масштаба до 0.5–1.0 поднимает алгоритм сразу на 13–16 пунктов.

popSize | pSwitch | levyScale | fromCode	Вариант	5 функций	25 функций	500 функций	Итог
50 | 0.8 | 0.01 | 1	код автора	0.422	0.223	0.140	26.16%
50 | 0.2 | 0.01 | 0	статья автора	0.479	0.259	0.145	29.41%
50 | 0.1 | 1.0 | 1	крупный шаг, база кода	0.770	0.255	0.132	38.57%
50 | 0.1 | 1.0 | 0	крупный шаг, база статьи	0.779	0.264	0.133	39.19%
50 | 0.0 | 0.5 | 0	только глобальный шаг	0.871	0.278	0.133	42.72%

Весь выигрыш сосредоточен в малой размерности: среднее по 5 функциям вырастает с 0.42 до 0.87. На 25 функциях все варианты лежат в узком коридоре, а на 500 функциях крупный шаг даже немного проигрывает мелкому. Разница между базой кода и базой статьи при крупном шаге в пределах шума: локальных шагов там всего 10%, и fromCode влияет только на них.

Почему большая размерность не поддаётся. Объяснение кроется в тяжёлом хвосте распределения Леви. На одну координату дальний прыжок выпадает редко, но глобальный шаг применяется ко всем координатам сразу. В таблице — сколько координат за один ход в среднем сдвигается больше чем на 3 и больше чем на 10 расстояний до лидера при levyScale = 1:

Размерность	Координат	Сдвиг больше 3 расстояний	Сдвиг больше 10 расстояний
5 функций	10	0.8	0.13
25 функций	50	3.8	0.6
500 функций	1000	76	12.5

На 10 координатах большинство ходов аккуратные, и редкий дальний прыжок по одной координате помогает выбраться из ловушки. На 1000 координатах в каждом ходе десятки координат улетают далеко и часто прижимаются к границе, так что такой ход почти всегда оказывается хуже исходной точки и отбрасывается. Уменьшение levyScale делает хвост безопасным, но и шаг становится слишком маленьким. Смелость и разрушительность растут вместе, и ни одно из проверенных значений параметра не дало сразу и то, и другое.

Локальное опыление работает только в малой популяции. Самый неожиданный результат касается оператора, который автор считал основным: в коде при p = 0.8 на него приходится 80% бюджета. Если локальный шаг выключить совсем (pSwitch = 0), результат не падает. Чистое сравнение есть только для 10 цветков:

popSize	pSwitch 0.1	pSwitch 0.2	pSwitch 0.0
50	40.54%	—	42.72%
20	44.97%	—	45.23%
10	—	49.16%	46.00%

(levyScale 0.5, fromCode 0; в прогонах с pSwitch 0.1 была включена ориентация разности от худшего к лучшему, см. ниже.)

Вероятная причина слабости локального шага в большой популяции в том, что он на самом деле не локальный. Разность двух случайных цветков в начале поиска сравнима с размером всей области, и этот ход оказывается самым крупным в алгоритме. Он меняет все координаты разом, а его направление связано с двумя случайными цветками, а не с тем, где находится сам цветок. При fromCode = 1 к этому добавляется старт от отвергнутой пробы.

В малой популяции картина меняется. При 10 цветках 20% локальных шагов дают 49.16% против 46.00% без них, и это больше типичного разброса между прогонами. Прибавка приходится на 25 и 500 функций: среднее по 25 функциям растёт с 0.414 до 0.487, по 500 — с 0.141 до 0.160. Когда цветков мало, их разности уже соизмеримы с окрестностью лучшего решения, и шаг вдоль разности двух соседей превращается в настоящую локальную доводку. В большой популяции те же разности, по-видимому, остаются крупными и только тратят бюджет.

Меньше цветков — лучше, но до предела. Без локального шага FPA сводится к набору независимых жадных поисковиков, которые общаются только через масштаб шага. Бюджет фиксирован (10 000 оценок), поэтому таблица сравнивает не размер популяции сам по себе, а соотношение числа цветков и числа итераций на каждый. Чем меньше цветков, тем больше итераций приходится на каждый, и на 25 функциях это даёт заметный рост (pSwitch 0, levyScale 0.5, fromCode 0):

popSize	5 функций	25 функций	500 функций	Итог
50	0.871	0.278	0.133	42.72%
20	0.864	0.357	0.136	45.23%
10	0.825	0.414	0.141	46.00%
5	0.767	0.476	0.147	46.35%
3	0.615	0.450	0.159	40.78%

Провал при трёх цветках объясняется особенностью глобального шага, о которой говорилось выше: лидер им не двигается. Его оценка каждую итерацию тратится впустую: стенд вычисляет фитнес всех агентов в каждой эпохе, в том числе точки, которая не изменилась. При 50 цветках это 2% бюджета, при 10 — 10%, при трёх — уже треть. К тому же окрестность лучшего решения — самое перспективное место — никто прицельно не исследует.

Проверенные правки, не вошедшие в итоговую версию. Две правки мы проверили и не стали включать в итоговую версию. Обе полезно знать тем, кто захочет сам доработать алгоритм.
Первая — ориентация локального шага от худшего к лучшему: из двух партнёров j становится лучшим, и разность всегда указывает в его сторону. Локальный шаг это не спасло: при pSwitch 0.1 результат всё равно не выше, чем без локальных шагов вообще. Вектор от x_k к x_j говорит, куда лучше двигаться рядом с этими двумя точками. Цветок i может сидеть в другой впадине, и там это направление почти ничего не значит.

Вторая — разморозка лидера: цветок, который держит лучшее решение, берёт масштаб глобального шага от расстояния до случайного соседа, а не до себя. При малой популяции она стабильно поднимает среднюю размерность, но итоговая оценка меняется в пределах разброса (pSwitch 0, levyScale 0.5, fromCode 0):

popSize	Без правки	С правкой	25 функций: без / с
50	42.72%	41.63%	0.278/0.286
10	46.00%	47.74%	0.414/0.444

popSize	Без правки	С правкой	25 функций: без / с
5	46.35%	46.97%	0.476/0.515

Правка помогает меньше, чем можно было бы ждать по освобождённому бюджету: расстояние до случайного соседа часто велико, и лидер встряхивается слишком грубо для доводки.

Итоговая конфигурация. От авторских параметров кода (26.16%) итоговая конфигурация отличается на 23 пункта. Средние по размерностям: 0.828 на 5 функциях, 0.487 на 25 и 0.160 на 500. Лучше всего алгоритм решает Forest на 5 функциях (0.939), хуже всего — Forest на 500 (0.074).

```
FPA|Flower Pollination Algorithm|10.0|0.2|0.5|0.0|
=============================
5 Hilly's; Func runs: 10000; result: 0.7853149391594381
25 Hilly's; Func runs: 10000; result: 0.5297058103938198
500 Hilly's; Func runs: 10000; result: 0.28144007940836896
=============================
5 Forest's; Func runs: 10000; result: 0.9385005907527866
25 Forest's; Func runs: 10000; result: 0.5048080878385968
500 Forest's; Func runs: 10000; result: 0.07367996881297378
=============================
5 Megacity's; Func runs: 10000; result: 0.7600000000000001
25 Megacity's; Func runs: 10000; result: 0.4253333333333333
500 Megacity's; Func runs: 10000; result: 0.12562666666666772
=============================
All score: 4.42441 (49.16%)
```

Проверка на честность. Хорошая цифра на стенде не всегда означает, что алгоритм так же хорошо сработает на параметрах советника. Он мог научиться пользоваться двумя особенностями стенда. Во-первых, все пары координат там одинаковы, и перенос значений между координатами или движение вдоль диагонали области дают дармовой выигрыш. Во-вторых, область поиска лежит около начала координат, и формула, которая тянет к нулю, неожиданно полезна.

У FPA оба оператора разностные: x_i − g в глобальном шаге и x_j − x_k в локальном. Число Леви берётся своё для каждой координаты, а единственный общий скаляр ε лишь сохраняет направление разности двух реальных решений и не переносит значения между координатами. Поэтому мы ждём, что оба теста покажут честный результат.

Оговоримся: эти тесты проверяют только две названные особенности стенда. Устойчивость к шуму, к разному масштабу и корреляции параметров, к ограничениям и нестационарности целевой функции они не проверяют.

Стенд со сдвигом. Область поиска сдвигается на 15 своих ширин, а перед вычислением функции сдвиг вычитается. Ландшафт остаётся прежним, меняется только положение начала координат: оно оказывается далеко за пределами области поиска. Алгоритм, чей результат здесь падает, опирался на положение нуля, а не на ландшафт. Для трейдера это прямо важно: диапазон вроде ценового уровня 1.0800–1.0900 лежит очень далеко от нуля относительно своей ширины.

```
FPA|Flower Pollination Algorithm|10.0|0.2|0.5|0.0|   ShiftFactor = 15.0
=============================
5 Hilly's; Func runs: 10000; result: 0.8452675640019767
25 Hilly's; Func runs: 10000; result: 0.5106539156036868
500 Hilly's; Func runs: 10000; result: 0.2800490332964667
=============================
5 Forest's; Func runs: 10000; result: 0.9117798655331132
```

```
25 Forest's; Func runs: 10000; result: 0.5270337901402008
500 Forest's; Func runs: 10000; result: 0.07346428068618865
==============================
5 Megacity's; Func runs: 10000; result: 0.7106666666666668
25 Megacity's; Func runs: 10000; result: 0.40186666666666665
500 Megacity's; Func runs: 10000; result: 0.12304000000000097
==============================
All score: 4.38382 (48.71%)
```

Композитный тест. Пары координат по очереди отдаются пяти разным функциям (Hilly, Forest, Megacity, Peaks, Skin) со своими диапазонами и положением оптимума, а фитнес цветка равен среднему по всем парам.

Любой выигрыш от одинаковости пар здесь исчезает. Если результат на композите заметно ниже результата на стенде в той же размерности, это признак того, что алгоритм пользовался одинаковостью пар. Обратное означает лишь отсутствие такого признака.

```
FPA|Flower Pollination Algorithm|10.0|0.2|0.5|0.0|
==============================
Composite anti-cheat test: Hilly + Forest + Megacity + Peaks + Skin
Pairs: 5; Coordinates: 10; Epochs: 1000; Repeats: 10
==============================
Run 1/10: 0.8546058922251879
Run 2/10: 0.8510964794303302
Run 3/10: 0.9809875328848362
Run 4/10: 0.8212067160441678
Run 5/10: 0.9839281875997858
Run 6/10: 0.8510964795233779
Run 7/10: 0.9866663121593137
Run 8/10: 0.9586587260729121
Run 9/10: 0.852831708770734
Run 10/10: 0.8528317077249306
==============================
Average result: 0.8993909742 (89.94%)
==============================
```

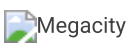
Визуализация работы алгоритма FPA.



FPA на тестовой функции Hilly



FPA на тестовой функции Forest



FPA на тестовой функции Megacity



FPA на функции Ackley

По результатам проведённых тестов алгоритм FPA представлен в рейтинговой таблице лучших популяционных методов ознакомительно.

cc	AO	Description	Hilly	Hilly Final	Forest	For Fin

			10 p (5 F)	50 p (25 F)	1000 p (500 F)		10 p (5 F)	50 p (25 F)	1000 p (500 F)	
1	ANS	across neighbourhood search	1,00000	0,88228	0,40138	2,28366	1,00000	0,95281	0,28092	2,2
2	AMOm	animal migration optimization M	0,91624	0,83603	0,46790	2,22017	0,98482	0,92010	0,36391	2,2
3	CLA	code lock algorithm (joo)	0,95139	0,86199	0,37879	2,19217	0,99349	0,93500	0,26497	2,1
4	(P+O)ES	(P+O) evolution strategies	0,86571	0,89539	0,39740	2,15850	0,97761	0,89820	0,26878	2,1
5	SDSm	stochastic diffusion search M	0,95195	0,84944	0,36249	2,16388	0,98061	0,88457	0,22112	2,0
6	AAm	archery algorithm M	0,84685	0,73320	0,42590	2,00595	0,96709	0,77837	0,27789	2,0
7	SIA	simulated isotropic annealing (joo)	0,93543	0,86504	0,38483	2,18530	0,94069	0,80609	0,23835	1,9
8	TETA	time evolution travel algorithm (joo)	0,91452	0,86369	0,25579	2,03400	0,99654	0,91291	0,14394	2,0
9	ESG	evolution of social groups (joo)	0,98111	0,79857	0,31167	2,09135	0,98954	0,82270	0,15032	1,9
10	CTA	comet tail algorithm (joo)	0,92435	0,86786	0,27838	2,07059	0,99039	0,84571	0,19448	2,0
11	COA	coyote_optimization_algorithm	0,88909	0,70681	0,32718	1,92308	0,99467	0,85358	0,15152	1,9
12	CSBO	circulatory_system_based_optimization	0,87793	0,79105	0,30025	1,96923	0,95026	0,88027	0,15364	1,9
13	ECBO	enhanced colliding bodies optimization	0,94024	0,72363	0,32356	1,98743	0,99477	0,80291	0,13056	1,9
14	BCO(B)	bezier_curve-based_optimization	0,99069	0,77836	0,30678	2,07583	0,98481	0,74554	0,13778	1,8
15	DA	dialectical algorithm	0,93117	0,75400	0,26205	1,94722	0,98925	0,81375	0,08662	1,8
16	BBO	biogeography based optimization	0,95876	0,70609	0,35752	2,02237	0,92981	0,70660	0,16970	1,8
17	BHAm	black hole algorithm M	0,79558	0,76207	0,34682	1,90447	0,99836	0,75798	0,13826	1,8
18	HS	harmony search	0,91420	0,69049	0,29924	1,90393	0,97627	0,73373	0,14193	1,8
19	RFO	royal flush optimization (joo)	0,80989	0,74481	0,34546	1,90016	0,95251	0,77926	0,15185	1,8
20	BOAm	billiards optimization algorithm M	0,76177	0,72421	0,25275	1,73873	0,90890	0,81960	0,28853	2,0
21	ASO	anarchy society optimization	0,73070	0,73713	0,31195	1,77978	0,99732	0,87700	0,17619	2,0
22	OOAm	osprey_optimization_algorithm	0,67372	0,68299	0,34866	1,70537	0,99999	0,86410	0,15774	2,0
23	EOm	extremal optimization_M	0,76527	0,75205	0,31908	1,83640	0,99999	0,76426	0,12437	1,8
24	ACS	artificial cooperative search	0,75545	0,77162	0,31653	1,84360	1,00000	0,80488	0,10705	1,9
25	SSG	saplings sowing and growing	0,75436	0,63206	0,35935	1,74577	0,91907	0,69694	0,19755	1,8
26	AOSm	atomic orbital search M	0,76184	0,68435	0,31344	1,75963	0,90015	0,80044	0,11501	1,8
27	TSEA	turtle shell evolution algorithm (joo)	0,95809	0,64852	0,29571	1,90232	0,99522	0,58104	0,10542	1,6
28	DE	flow_direction_algorithm	0,96398	0,62346	0,26089	1,84833	0,98482	0,77018	0,11459	1,8
29	BIO	blood inheritance optimization (joo)	0,72580	0,66522	0,31228	1,70330	0,99995	0,68125	0,11540	1,7
30	(PO)ES	(PO) evolution strategies	0,73972	0,58190	0,38896	1,71058	0,91199	0,59975	0,21262	1,7
31	BO	bonobo optimizer	0,75555	0,64366	0,32657	1,72578	0,94332	0,70442	0,13999	1,7
32	SRA	successful restaurateur algorithm (joo)	0,89010	0,63359	0,29115	1,81484	0,96634	0,55285	0,08914	1,6
33	FDAm	flow_direction_algorithm_M	0,87573	0,58806	0,27135	1,73514	0,99997	0,64399	0,09633	1,7
34	CRO	chemical reaction optimisation	0,91281	0,65681	0,29866	1,86828	0,90513	0,56020	0,10939	1,5
35	BCOm	bacterial chemotaxis optimization M	0,82589	0,61733	0,31584	1,75906	0,95296	0,63718	0,11984	1,7
36	DOA	dream optimization algorithm	0,78522	0,78121	0,36036	1,92679	0,61584	0,42117	0,12254	1,1
37	ABO	african buffalo optimization	0,92295	0,62528	0,29885	1,84708	0,92992	0,57468	0,09372	1,5
38	BSA	bird swarm algorithm	0,94432	0,67941	0,26401	1,88774	0,91649	0,65619	0,12054	1,6
39	TSm	tabu search M	0,87806	0,61040	0,28993	1,77839	0,98116	0,52165	0,08544	1,5
40	BSA	backtracking search algorithm	0,87128	0,53190	0,28675	1,68993	0,92408	0,51602	0,09153	1,5
41	BEA	bacterial_evolutionary_algorithm	0,92170	0,59615	0,29340	1,81125	0,96906	0,44500	0,08233	1,4
42	BWOm	beluga_whale_optimization_M	0,78488	0,56872	0,29557	1,64917	0,91370	0,61760	0,12988	1,6
43	WOAm	whale optimization algorithm M	0,93893	0,59477	0,26695	1,80065	0,98036	0,53873	0,07112	1,5
44	ACA	andean_condor_algorithm	0,78444	0,53260	0,33108	1,64812	0,79071	0,44960	0,10685	1,3
45	CSO	competitive swarm optimizer	0,85151	0,60786	0,29896	1,75833	0,84085	0,58491	0,11974	1,5
	FPA	flower_pollination_algorithm	0,78531	0,52970	0,28144	1,59645	0,93850	0,50480	0,07368	1,5

	RW	random walk	0,49970	0,32333	0,25791	1,08094	0,30754	0,11470	0,04400	0,4

Выводы

С параметрами автора FPA находится на уровне случайного поиска: 26% по коду и 29% по статье. Трейдер, который возьмёт его из библиотеки как есть, получит параметры советника, найденные почти наугад. Причина не в идее алгоритма, а в двух числах: множитель 0.01 делает глобальный шаг неподвижным, а p = 0.8 отдаёт большую часть бюджета локальному шагу, который в большой популяции ничего не показал.

После перенастройки алгоритм теряет большую часть своей метафоры. Популяция сжимается до десяти цветков, и почти всю работу делает один оператор — встряска с тяжёлым хвостом, размах которой задаёт расстояние до лидера. Итоговая оценка — 49.16%

Для оптимизации торговых систем это означает следующее. На советниках с десятком параметров настроенный FPA будет работать хорошо и приблизится к глобальной области. На десятках параметров он ещё пригоден, если держать популяцию маленькой. На сотнях параметров — например, при обучении весов небольшой сети — его брать не стоит: там он не сильнее случайного поиска при любых настройках, и лучше выбрать алгоритм из верхней части таблицы.

И второй урок, который важнее самого FPA. Перед тем как доверить оптимизатору свой советник, стоит прогнать его хотя бы на двух-трёх значениях каждого параметра и посмотреть, что из него действительно работает. У FPA такая проверка дала больше, чем любая модификация.



Рисунок 2. Цветовая градация алгоритмов по результатам соответствующих тестов



Рисунок 3. Гистограмма результатов тестирования алгоритмов (по шкале от 0 до 100: чем больше, тем лучше, где 100 — максимально возможный теоретический результат). В архиве приведён скрипт для расчёта рейтинговой таблицы

Плюсы и минусы алгоритма FPA:

Плюсы

- Очень простой алгоритм: два оператора, никаких сортировок и расписаний.
- Хорошие результаты в малой размерности, в том числе на ступенчатой Megacity.
- Оба оператора разностные, без формул, зависящих от начала координат или диагонали области.

Минусы

- Авторские параметры дают уровень случайного поиска, алгоритм нужно перенастраивать.
- Локальное опыление полезно только в малой популяции: при 20–50 цветках пользы от него не обнаружено.
- На 1000 координатах результат не выше случайного поиска при любых настройках.

К статье прикреплён архив с актуальными версиями кода алгоритмов. Автор статьи не несёт ответственности за абсолютную точность в описании канонических алгоритмов, во многие из них внесены изменения для улучшения поисковых возможностей. Выводы и суждения, представленные в статьях, основываются на результатах проведённых экспериментов.

- github: https://github.com/JQSakaJoo/Population-optimization-algorithms-MQL5

Программы, используемые в статье

#	Имя	Тип	Описание
1	#C_AO.mqh	Включаемый файл	Родительский класс популяционных алгоритмов оптимизации
2	#C_AO_enum.mqh	Включаемый файл	Перечисление популяционных алгоритмов оптимизации
3	TestFunctions.mqh	Включаемый файл	Библиотека тестовых функций
4	TestStandFunctions.mqh	Включаемый файл	Библиотека функций тестового стенда
5	TestStand3D.mqh	Включаемый файл	3D-панель визуализации для тестового стенда
6	Utilities.mqh	Включаемый файл	Библиотека вспомогательных функций
7	CalculationTestResults.mqh	Включаемый файл	Скрипт для расчёта результатов в сравнительную таблицу
8	Test_AO_All.mq5	Скрипт	Единый испытательный стенд для всех популяционных алгоритмов оптимизации
9	Test_AO_AntiCheat	Скрипт	Тест на читерство алгоритмов оптимизации
10	Simple use of population optimization algorithms.mq5	Скрипт	Простой пример использования популяционных алгоритмов оптимизации без визуализации
11	Test_AO_FPA.mq5	Скрипт	Испытательный стенд для FPA

Следующая статья >>

Andrey Dik

Россия 75797

РАССМОТРЮ ПРЕДЛОЖЕНИЯ ПО ИЗДАНИЮ КНИГИ (УЧЕБНОГО ПОСОБИЯ) ОБ АЛГОРИТМАХ ОПТИМИЗАЦИИ.
Группа для общения по вопросам оптимизации и бесплатного тестирования продуктов: https://t.me/+vazsAAcney4zYmZi
Внимание! Появились мои двойники в телеграм, мой настоящий ник @JQS_aka_Joo
Мой github с алгоритмами оптимизации: https://github.com/JQSakaJoo/Population-optimization-algorithms-MQL5
Все мои публикации: https://www.mql5.com/en/users/joo/publications
Разрабатываю системы по технологиям машинного обучения с 2007 года и в области искусственного интеллекта, оптимизации и прогнозирования.

Другие статьи автора

- Оптимизация на основе кривых Безье — Bezier Curve-based Optimization (BCO)
- Алгоритм оптимизации скопы — Osprey Optimization Algorithm(OOA)
- Эволюция городских советов — City Councils Evolution (CCE)
- Алгоритм оптимизации на основе кровеносной системы — Circulatory System Based Optimization (CSBO)
- Алгоритм оптимизации на основе приспособленности — Fitness Dependent Optimizer (FDO)
- Алгоритм направления потока — Flow Direction Algorithm (FDA)

Подпишитесь и узнавайте о
популярных статьях и кодах